Zoeken tot beperkte diepte met heuristiek

Bas Terwijn

March 1, 2026

1 Introductie

Dit document behandelt zoeken tot beperkte diepte met heuristiek in de Tictactoe game.

1.1 Doelen

Bekend raken met:

- Zoeken tot beperkte diepte
- Heuristieken

2 Zoeken tot beperkte diepte

Als we twee `Player_AI_AllMove` spelers tegen elkaar laten spelen in de Tictactoe game met:

```
$ python play.py -p1 aa -p2 aa
```

Dan denkt elke speler all mogelijke zetten vooruit en verliest daarmee nooit. Dat is prima voor een klein spel zoals Tictactoe, want het aantal mogelijke zetten is nog enigsinds beperkt. Bij de eerste zet zijn er 9 mogelijke zetten, bij de tweede zet 8 mogelijke zetten, enzovoorts. Het totale aantal mogelijke spelverlopen is dus $9! = 362.880$, wat nog steeds door een computer in redelijke tijd kan worden doorgerekend.

Maar, als we het Tictactoe bord 4x4 in plaatse van 3x3 groot maken met:

```
$ python play.py -s 4 -p1 aa -p2 aa
```

dan gaat het totale aantal mogelijke spelverlopen dus naar $16! = 20.922.789.888.000$, wat veel te groot is om in redelijke tijd door een computer te laten doorgerekend. We zouden bij Tictactoe nog met gebruik van symmetrie het aantal zetten dat geëvalueerd moet worden kunnen verkleinen, maar dat is niet in alle spellen mogelijk of voldoende.

Een betere oplossing is om niet alle mogelijke zetten vooruit te denken, maar slechts tot een bepaalde diepte. Bijvoorbeeld, we kunnen slechts 3 zetten vooruit denken. Dat betekent dat we op een 4x4 board bij de eerste zet 16 mogelijke zetten hebben, bij de tweede zet 15 mogelijke zetten, en bij de derde zet 14 mogelijke zetten. Het totale aantal zetten dat we dan moeten evalueren is dus $16 \times 15 \times 14 = 3360$, wat wel snel door te rekenen is.

Hiervoor hebben we `Player_AI_DMove` geïmplementeerd, waarbij de `D` staat voor *Depth* die we kunnen instellen. Deze class is te vinden in file `player_ai_dmove.py`.

2.1 `Score` class

Daarnaast willen we als een speler wint of verliest ook bijhouden op welke diepte dat gebeurt. Als een speler bijvoorbeeld na 2 zetten wint, dan is dat beter dan wanneer die speler pas na 3 zetten wint. We kunnen dus een score toekennen op basis van de diepte waarop een speler wint of verliest. Voor dit doel is de `Score` class aan `player_ai_dmove.py` toegevoegd.

De `Player_AI_DMove` werkt al. Lees de implementatie van de class door en test deze class met:

```
$ pytest test_player_ai_dmove.py -vv
```

Het is nu wel mogelijk om op een 4x4 bord te spelen. We kunnen bijvoorbeeld een `Player_AI_DMove` speler die 3 ('d3') zetten vooruit denkt, laten spelen tegen een `Player_AI_DMove` speler die 2 ('d2') zetten vooruit denkt:

```
$ python play.py -s 4 -p1 d3 -p2 d2
```

Om te zien hoe nu een zet wordt gekozen runnen we als voorbeeld:

```
$ python recursion_tree.py
```

welke de zoekboom van zetten laat zien van een `Player_AI_DMove` speler die maximaal 1 zet vooruit denkt. Een nadeel van zo vroeg stoppen is dat veel zetten nu een `Score(0, 0, 0)` score krijgen, omdat de speler niet ver genoeg vooruit heeft kunnen denken om te zien of een zet goed of slecht is. Om dat op te lossen kunnen we een heuristiek gebruiken.

2.2 Opdracht `tictactoe_heuristic.py`

Een heuristiek is een functie die een schatting geeft van hoe goed een bepaalde bordsituatie is. In Tictactoe kunnen we bijvoorbeeld kijken naar het aantal mogelijke winnende lijnen dat een speler nog heeft. Hoe meer winnende lijnen een speler nog heeft, hoe beter de situatie voor die speler is. Een 3x3 Tictactoe bord heeft, zoals weergegeven in figuur 1, 3 horizontale (blauw), 3 verticale (groen) en 2 diagonale (rood) winnende lijnen, dus in totaal 8 winnende lijnen.

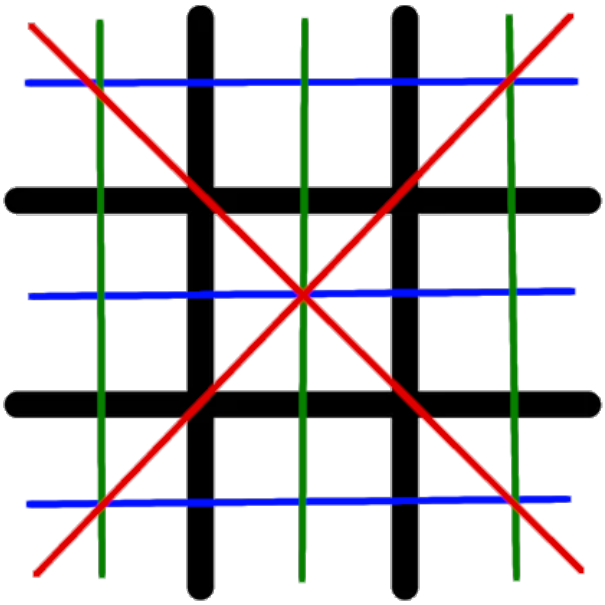

Figure 1: Tictactoe bord met alle mogelijke winnende lijnen

We gaan voor iedere lijn het aantal punten bepalen voor een speler. We nemen hier speler 'X' als voorbeeld met tegenspeler 'O', maar hetzelfde geldt andersom.

- als een lijn geen 'O' bevat, dan is het aantal punten gelijk aan het aantal 'X' op die lijn, bijvoorbeeld 'X - X' geeft 2 punten.
- als een lijn geen 'X' bevat, dan is het aantal punten gelijk aan het negatieve aantal 'O' op die lijn, bijvoorbeeld 'O - O' geeft -2 punten.

- als een lijn zowel 'X' als 'O' bevat of helemaal leeg is, dan is het aantal punten gelijk aan 0, bijvoorbeeld 'X - O' geeft 0 punten.

De heuristiek voor een bord voor speler 'X' is dan de som van de punten van alle lijnen, dus de som 2 voor het bord in figuur 2. De heuristiek voor speler 'O' is de negatieve waarde hiervan, dus de som -2.

Figure 2: Tictactoe bord met heuristiek van 2 voor speler 'X' en -2 voor speler 'O'

Een heuristiek is een schatting van hoe goed een bepaalde bordsituatie is en die kan op veel verschillende manieren worden gemaakt, we kiezen hier met enige willekeur voor deze aanpak. Implementeer deze heuristiek in file `tictactoe_heuristic.py` in functies:

```
def estimate_line_score(tictactoe, player_value, positions)
```

Test uw implementatie met:

```
$ pytest test_tictactoe_heuristic.py -vv
```

Als de heuristiek goed werkt, dan wordt nu in het eerdere voorbeeld wel altijd de winnende zet gekozen als gevolg van de heuristiek:

```
$ python recursion_tree.py
```

Om zelf tegen een `Player_AI_DMove` speler te spelen die 4 zetten vooruit denkt met de heuristiek ('d4h') gebruiken we:

```
$ python play.py -p1 h -p2 d4h
```